

DATA DRIVEN TESTING

Testing the application with multiple data which is kept in external resource files like excel, properties file, etc. is called Data Driven Testing.

Advantages of Data Driven Testing

1. Maintenance of test data in excel or properties file is easier.
2. Modification of test data is easier.
3. Reusability of test data.
4. Test data can be created before the test execution.
5. We can test the application with huge volume of data.

Properties File

- Properties file is used to store the common data.
- Here, data will be stored in the form of key-value pair.
- Key and value should be separated by “ : ”.
- By default, all the data available in properties file is a String
- It has the extension “**.properties**”.

Advantages of Properties File

1. It is very fast in execution.
2. It is very light-weight compared to any other external resource file.

Disadvantages of Properties file

- Test data cannot be stored in well- mannered form.
- In one line, we can store only one key-value pair.
- Each and every data should have unique key.
- When we load the properties file, all the key-value pair will be loaded into that properties object.
- Modification of the data is difficult.

Steps to Fetch Data From Properties file

Step 1:-

Create an object of FileInputStream

```
FileInputStream fis= new FileInputStream("File Path");
```

Step 2:-

Create file type Object

```
Properties prop= new Properties();
```

Step 3:-

Call the read methods

```
prop.load(fis);
```

```
prop.getProperty("key");
```

Excel File

- We store test script data in excel file.
- Whenever we want to read data from Excel, we need to use Apache poi plugin so that we can read the data from all the Microsoft Documents like **.xls** and **.xlsx**.
- Apache poi is a free and open source library tool similar to Selenium.

Steps to Install Apache Poi

1. Go to “apache poi download”
2. Click on first link and scroll down till Binary artifacts and click on it.
3. Click on poi-bin zip file.
4. Extract zip file
5. In eclipse, create a folder “Jars” under the project.
6. Copy the jars present in “lib”, “ooxml-lib”, and “poi-bin” folders and paste it in “Jars” folder in eclipse.
7. Add all the jars to build path.

Steps to Fetch Data from Excel File

Step 1:-

Create an object of FileInputStream

```
FileInputStream fis= new FileInputStream("File Path");
```

Step 2:-

Create file type Object

```
Workbook wb= WorkbookFactory.create(fis);
```

Step 3:-

Call the read methods (Using workbook reference)

To fetch data in String format:-

```
wb.getSheet("SheetName").getRow(index).getCell(index).toString();
```

OR

```
wb.getSheet("SheetName").getRow(index).getCell(index).getStringCellValue();
```

To fetch data in Numerical format:-

```
wb.getSheet("SheetName").getRow(index).getCell(index).getNumericCellValue();
```

To fetch data in Date format:-

```
wb.getSheet("SheetName").getRow(index).getCell(index).getLocalDateTimeCellValue();
```

To fetch data in Boolean format:-

```
wb.getSheet("SheetName").getRow(index).getCell(index).getBooleanCellValue();
```

TESTNG

TestNG Stands for “Test Next Generation”.

- It is a unit testing framework tool comes under TDD[Test Driven Development] framework, which supports Java and .net
- It is an open source unit testing framework, where NG stands for Next Generation
- TestNg is developed as an additional plugin for Eclipse.
- TestNG is inspired from JUnit & Nunit
- Basically used by developers to perform unit testing.
- We automation engineers use TestNG for –
 1. Batch Execution
 2. Parallel Execution
 3. Group Execution
 4. Report Generation
 5. Assertion
 6. Cross Browser Testing
 7. Easy Execution

Advantages of TestNG

- I. TestNG is used to execute Batch execution without any manual interaction.
- II. TestNG handles the framework component.
- III. It Produces HTML Report after execution.
- IV. It has some additional functionality like Listeners(To identify failed Test Script and take screenshot)
- V. In Selenium automation, TestNG acts like main controller of the framework.
- VI. TestNG also can be used to achieve parallel execution and group execution.

Rules to follow while using TestNG

1. Use reporter.log() instead of printing statement
2. Use @Test instead of main method

Installation of TestNG

1. Open eclipse and click on “help” major tab
2. Select “Eclipse Marketplace”.
3. Search “TestNG” in eclipse marketplace dialogue box
4. Click on “install” button under “TestNG for Eclipse”
5. Click on “confirm”
6. Click on “I agree to license agreement” radio button and then “next”
7. Select “install anyway” and then finish
8. Click on “restart now”
9. Right click on your project and click “build path”
10. Select “Configure Build Path”
11. In Library section, click “Add library”
12. Click on “TestNG” and then “next”, “next” and then “Apply&Close”

Batch Execution

Executing a set of test cases one after another via testing.xml file is called as Batch Execution. xml file is also called as suite file.

We Create suite file to do Batch Execution.

To do Batch Execution, we do:

- Select the test cases and right click.
- Select “TestNG” and then “Convert To TestNG”.
- Give the suite file name and then finish.
- Open the suite file/ xml file and Run.

Parallel Execution

Executing a set of test cases parallely is called Parallel Execution.

To do Parallel Execution, we do:

- Create a suite file with the set of test cases.
- Open the suite file.
- In the test level write the following-

`<test parallel=“classes” or
 =“methods” or
 =“true” or
 =“tests”`

Group Execution

To execute the test cases based on the group it belongs to is called as Group Execution.

To do Group Execution, we do-

- Create a suite file.
- Open the suite file.
- After the test level write the following-

```
<test>
    <groups>
        <run>
            <include name="smoke suite name"></include>
        </run>
    </groups>
```

We can use <exclude> tag to skip the execution of particular group of test cases.

Report Generation

TestNG provides report after every execution automatically.

- Refresh the project after the execution.
- Go to "test-output" folder
- 2 types of reports will be available in this folder
-index.html and emailableReport.html
- Right click on emailableReport.html
- View in system editor.

Note:

Every time we run script, old execution report will be replaced with new execution report.

Annotations

- Annotation is a java template which is used to give information to JRE, compiler and end users.
- It always starts with @ symbol.
- It can be used on top of Variables, Methods or Classes.
- There are 3 categories-
 1. Basic Annotations
 2. Configuration Annotations
 3. Parameter Annotations

Basic Annotations

@Test

@BeforeMethod

@AfterMethod

@BeforeClass

@AfterClass

Configuration Annotations (Additional annotations available only in TestNG)

@BeforeTest
@AfterTest
@BeforeSuite
@AfterSuite

Parameter Annotations

@Parameters
@DataProvider
@Listeners

@Test

- It is the core annotation of TestNG, without which no TestNG Programs will run.
- It is responsible to trigger the execution.
- Any Class which contains at least one @Test annotated method is called as TestNG Class.
- Any Method which is annotated with @Test annotation is considered as a Test Case.
- Any Code written within this method is considered as Test Steps.

Note:

TestNG follows alphabetical order of execution if there are more than 1 @Test core annotation.

Helper Attributes/ Annotation Attributes

1. Priority
2. InvocationCount
3. ThreadPoolsSize
4. Enabled
5. DependsOn
6. DataProvider
7. Parameters

These are attributes which are going to help us from the execution perspective.

Priority

- It determines the order of execution.
- We can give both positive and negative values.
- Least the value, first the execution.
- If Priority is not specified, then the default value is 0.
- Eg:- @Test(priority= -1)

Note:

We go for priority to execute the test cases in required order.

If priority is duplicated, then it will be executed in alphabetical order.

InvocationCount

- It is used to execute same test case multiple times with same test data.
- We can give positive integer values.
- Default value is 1.
- Eg:- @Test(invocationCount= 10)

ThreadPoolSize

- It is used to execute same test case multiple times with same test data simultaneously.
- In order to use this, invocation count is mandatory.
- By default, threadPoolSize is 0.
- Eg:- @Test(invocationCount=10, threadPoolSize= 5)

Enabled

- It is used to skip the test cases.
- We can give Boolean values(True/False).
- Default value is true.
- If you want to skip some test cases, then you need to explicitly specify "false".
- Eg:- @Test(enabled= false)

DependsOnMethod

- It is used when one test case is dependent on another test case (It means there is a dependency between methods).
- It will wait till the dependent method is executed successfully and then only execute that particular method.
- The scope of dependsOnMethods is within particular TestNG Class.
- Eg:-

```
@Test(dependsOnMethods= "createAccount")
public void deleteAccount() {
    Reporter.log("Account deleted Successfully");
}
@Test
public void createAccount(){
    Reporter.log("Account created Successfully");
}
```

Chronological Order of configuration annotations

```
@BeforeSuite
  @BeforeTest
    @BeforeClass
      @BeforeMethod
        @Test (test case TC_001)
          @AfterMethod
            @AfterClass
              @AfterTest
                @AfterSuite
```

@BeforeSuite

It will run only once for a suite file.

Here, we write code for establishing the Database Connection and to configure report.

@AfterSuite

Here, we disconnect the Database Connection and flush all the reports.

@BeforeTest

It will be executed based on the number of test runners in a suite file.

Preconditions which is common for a particular test runner will be written here.

@AfterTest

Post conditions which is common for that particular test runner will be written here.

@BeforeClass

It will execute for each class tag in a test runner.

Here, we write code to launch the Browser.

@AfterClass

Here, we write the code to close the Browser and the session.

@BeforeMethod

It will execute for each test case. i.e., for each @Test method.

Here, we write code for login actions.

@AfterMethod

Here, we write code for logout Actions.

Data Provider

- To execute the same test case multiple times with different set of test data, we make use of @DataProvider annotation.
- Sending data from data provider to test case is called dependency injection.
- Eg:-

```
@DataProvider(name="credential")
public String [] [] testData(){
    String [][] data= {{"admin", "manager"}, {"trainee", "trainee"}};
    return data;
}
```

```
@Test(dataProvider= "credential")
public void getData(String[] cred){
    System.out.println(cred[0]);
    System.out.println(cred[1]);
}
```

Output:

admin

Assertion

- Assertions are basically the libraries or functions which are used to verify certain critical checkpoints of test cases.
- Asserts in Selenium validate the automated test cases that help testers understand if tests have passed or failed.
- We go for Assertion to reduce the lines of code.
- There are 2 types-
 1. Hard Assert
 2. Soft Assert

1. Hard Assert

- If there is any failure, it will terminate the execution abruptly.
- Use assertEquals() present in Assert class.
- Syntax:-

```
Assert.assertEquals(expected value, actual value, message);
```

2. Soft Assert

- If there is nay failure, it will not terminate the execution
- Use assertEquals() present in SoftAssert Class.
- Syntax:

```
SoftAssert assert= new SoftAssert();  
assert.assertEquals( expected value, actual value, message);
```
- Note:- At the end of the script call assertAll() to show assertion failure().

Hard Assert	Soft Assert
In case of failure, test cases will be terminated abruptly.	It will not hard stop the execution. It will continue the entire test case execution even when there is a failure.
No need to call any methods to show assertion error.	We need to call <code>assertAll()</code> to show assertion failure.
All the methods are static methods.	All the methods are non static methods.
It can be used for critical scenarios.	It can be used for non-critical scenarios.

Page Object Model(POM)

- POM is one of the java design pattern which is used to store the elements.
- We write separate java class for each and every page of the web application.
- It is also called as element repository of a particular web page.
- It is also used to avoid **StaleElementReferenceException**.
- In POM, we declare the element by using **@FindBy** annotation.
- We initialize the element by using **PageFactory.initElements()** method

Note:-

PageFactory.initElements() method will take 2 arguments

1. WebDriver (driver)
2. Object of POM class or page class (this)

To find element:

Syntax:-

```
@FindBy ( locator name= "locator value")  
private WebElement elementName;
```

Note:- Provide getter method.

Rules of POM:

1. Number of web pages in an application should be equal to number of POM classes.
2. Number of web elements present in a web page should be equal to Number of web element variables.
3. We should make the variables private.
4. We should access them only through getters.
5. Initialize elements with the help of PageFactory Class .

Advantages of POM

1. Maintenance of web elements is easy, since web elements are maintained based on the page.
2. Modification of web element is easy whenever the UI is changed.
3. We can avoid StaleElementReferenceException.
4. Object Repositories can be easily shared across the team members via GitHub.